Final Project Report

E-Commerce Shopping Cart

Allazhar Kuat
Akylzhan Khafiz
Yerkhan Dautbekov
Group SE-2515

1 Introduction

We built a console application for buying configurable laptops. A customer can choose RAM, storage and color, add services, put products in a cart, select a discount and complete checkout. Price and stock changes notify subscribed observers immediately.

Our goal is to show six design patterns working in one purchase flow. We keep the application in memory and run its operations sequentially so the relationships between the objects remain easy to follow. Payment is simulated through a service with an interface different from the one used by checkout.

Assigned responsibility	Patterns	Main area
Allazhar Kuat	Strategy and Facade	Discounts, cart and checkout
Akylzhan Khafiz	Builder and Decorator	Product configuration and services
Yerkhan Dautbekov	Observer and Adapter	Catalog notifications and payment

Application scope

The catalog contains Campus and Pro laptop models. Stock is tracked by base model identifier, while RAM upgrades, storage upgrades and services add to the current base price. All monetary values use BigDecimal in KZT. The menu supports adding and removing lines, changing quantities, selecting discounts, changing catalog values and checkout.

Repository and execution

github.com/ErknDt/SDP_Final_Project

```
mvn verify
java -jar target/SDP_Final_Project-1.0-SNAPSHOT.jar --demo
```

2 Product configuration and services

2.1 Builder

Laptop.Builder collects the model identifier, model name, RAM, SSD capacity and color. The defaults are 8 GB RAM, 256 GB SSD and Silver. build() validates the complete configuration and creates an immutable Laptop. Reusing the builder to create another configuration preserves the values of a previously built laptop.

```
PurchasableItem laptop = new Laptop.Builder("CAMPUS", "Campus Laptop")
        .ramGb(16).storageGb(512).color("Black").build();
```

The constructor accepts RAM values 8, 16 and 32 GB, and SSD values 256, 512 and 1024 GB. Unsupported values and blank text are rejected. RAM and SSD choices determine the configuration surcharge. The catalog supplies the current base model price.

Trade-offs. The builder adds a separate construction object. Validation happens when build() is called, so invalid intermediate values can exist in the builder. Adding a configuration option requires maintaining the builder and the resulting product.

2.2 Decorator

Laptop and the service wrappers implement PurchasableItem. ProductDecorator stores a reference to another PurchasableItem. GiftWrapDecorator adds 3000 KZT and ExtendedWarrantyDecorator adds 25000 KZT. Each wrapper extends the description and delegates to the wrapped item when calculating its surcharge.

```
laptop = new ExtendedWarrantyDecorator(new GiftWrapDecorator(laptop));
```

A 16 GB, 512 GB configuration adds 35000 KZT. With both services its total surcharge is 63000 KZT. Wrappers preserve the base product identifier, which lets checkout aggregate stock across differently configured lines of the same model.

Trade-offs. Several small wrapper classes are needed. A long wrapper chain takes more effort to inspect. Service combinations need a defined policy, because wrapping a product twice with the same service would charge that service twice. The interactive menu offers each service once per configured line.

UML interpretation

The product class diagram appears in Section 7. Dashed hollow-triangle arrows mean interface realization. Solid hollow-triangle arrows mean inheritance. The wrapped association shows the object reference held by ProductDecorator.

3 Catalog updates and payment integration

3.1 Observer

ProductCatalog is the subject. It stores a separate subscriber list for each product. After a successful update, it calls ProductObserver.onProductChanged with the identifier, current base price and stock. ConsoleNotificationObserver displays every update. LowStockObserver displays a warning when stock reaches its configured threshold.

```
catalog.subscribe("CAMPUS", notifications);
catalog.subscribe("CAMPUS", lowStock);
catalog.updatePrice("CAMPUS", new BigDecimal("280000.00"));
```

Notifications are synchronous. The catalog iterates over a copy of the subscriber list so a callback can unsubscribe safely. An observer runtime exception is logged and delivery continues. Bulk stock changes validate every requested product before updating any stock, then notify subscribers after all updates are committed.

Trade-offs. Synchronous callbacks add to update latency. Subscriptions need lifecycle management. Callback failures and changes to subscriber lists need explicit handling. Our tests cover duplicate subscription, removal and a failing observer.

3.2 Adapter

Checkout uses PaymentGateway.pay(BigDecimal). LegacyPaymentService instead accepts a long amount in tiyn plus a currency code and returns a string status. PaymentAdapter converts the decimal amount to exact minor units, supplies KZT and converts APPROVED into true. Invalid precision and numeric overflow are rejected before charging.

```
minorUnits = validAmount.movePointRight(2).longValueExact();
return "APPROVED".equals(service.charge(minorUnits, "KZT"));
```

Trade-offs. The adapter is coupled to the external calling convention. Status and currency mappings require maintenance when that convention changes. Its conversions must be checked carefully to avoid rounding or overflow errors.

Payment scope

The service simulates approval and decline. A menu command declines the next attempt so the application can demonstrate recovery. Both the catalog and payment service are local objects, and the application performs no network payment.

4 Discounts and checkout coordination

4.1 Strategy

ShoppingCart is the context and stores a DiscountStrategy reference. NoDiscountStrategy returns zero, PercentageDiscountStrategy calculates a percentage, and ThresholdDiscountStrategy applies a fixed discount from a specified subtotal. The client replaces the strategy on the same cart while the application is running.

```
cart.setStrategy(new PercentageDiscountStrategy(10));
CartQuote quote = cart.quote(catalog);
```

The shared method returns a discount amount. The cart subtracts it from the subtotal. A fixed discount is capped at the subtotal, percentage results use HALF_UP rounding, and the cart rejects an invalid strategy result. A new quote reads current catalog prices. Previously returned quotes preserve their original prices and totals.

Trade-offs. The client needs to choose a suitable strategy. Separate strategy classes add structure to a small calculation. Every strategy must fit the shared input and result contract, so a rule requiring additional customer data would need a deliberate interface extension.

4.2 Facade

CheckoutFacade exposes placeOrder(cart). It obtains a quote, aggregates requested quantities by product identifier and validates stock before payment. Approval leads to stock reduction and cart clearing. A decline returns an unsuccessful OrderResult and preserves the cart for retry. The order result contains the quote used for payment.

```
if (!payment.pay(quote.total())) {
    return new OrderResult(false,
            "Payment declined. The cart is ready for another attempt.", quote);
}
catalog.decreaseStock(quantities);
cart.clear();
```

Trade-offs. A facade depends on several subsystem contracts. Its public operation can grow if unrelated responsibilities are added. It simplifies access but still relies on the subsystem guarantees. We keep validation and calculations in their own classes and use the facade to coordinate the purchase sequence.

5 Clean Code decisions

We used principles discussed in Chapters 16 and 17 of Clean Code. The examples below refer to the implementation and regression checks in this repository.

Descriptive names

Names such as getAdditionalPrice, amountInTiyn and decreaseStock describe both the operation and the units. DiscountStrategy.calculateDiscount explicitly returns the amount to subtract. This helps prevent confusing a discount with the final payable total.

Polymorphism for replaceable behavior

ShoppingCart delegates to DiscountStrategy. CheckoutFacade uses PaymentGateway, and ProductCatalog calls ProductObserver. These clients use their shared contracts without selecting implementations through class-name checks.

```
BigDecimal discount = Money.requireValid(strategy.calculateDiscount(subtotal));
```

Avoid duplicated validation

Money.requireValid centralizes checks for null values, negative values and excessive decimal precision. Catalog prices, cart surcharges, discounts and payments use the same rule. Percent rounding remains in the strategy that performs the calculation.

Keep operations focused

The builder constructs a configuration, decorators add services, strategies calculate discounts and the adapter translates payment calls. Console input and output are handled by ShopApplication and the display observers. CheckoutFacade coordinates the existing operations.

Control access to mutable data

Laptop fields are final. Cart items are returned through List.copyOf. CartQuote copies its list and stores immutable OrderLine values, including the price used for that quote. Internal catalog entries and subscriber lists stay private.

```
public List<CartItem> getItems() {
    return List.copyOf(items);
}
```

Use boundary and regression tests for safe changes

The tests include a rounding boundary, an exact discount threshold, a declined payment, insufficient aggregate stock, observer failure and numeric overflow. They provide known behavior to compare after refactoring. Each test creates its own objects, directly or through BeforeEach setup.

6 Verification and observed results

The verified Maven run completed **49 tests with zero failures, errors or skipped tests**. The runnable JAR was also executed with --demo. The interactive menu is exercised through scripted input in ShopApplicationTest.

Test class	Tests	Main checks
ShopApplicationTest	3	Complete demo, menu purchase, invalid input and EOF
ShoppingCartTest	8	Current prices, strategy replacement, protected snapshots
ProductCatalogTest	11	Delivery, unsubscribe, bulk update and observer failure
CheckoutFacadeTest	8	Success, decline/retry, stock, reentrancy and overflow
DiscountTest	8	Thresholds, rounding, bounds and invalid arguments
PaymentAdapterTest	5	Exact conversion, decline, invalid amount and overflow
ProductTest	6	Builder validation, snapshots and decorator composition

Reproducible purchase

Step	Observed amount or state
Configure and wrap two Campus laptops	2 x (300000 + 35000 + 28000) = 726000 KZT
Apply threshold discount	726000 - 50000 = 676000 KZT
Replace it with a 10% strategy	726000 - 72600 = 653400 KZT
Change the base price to 280000 KZT	686000 - 68600 = 617400 KZT
Decline payment	Stock stays 5 and the cart retains its line
Retry with approval	Stock becomes 3 and the cart becomes empty

PaymentAdapter sends 61740000 tiyn for the approved 617400.00 KZT order. The price update produces a product notification. The stock reduction produces another notification and a low-stock warning at the threshold of three units.

7 UML - product configuration

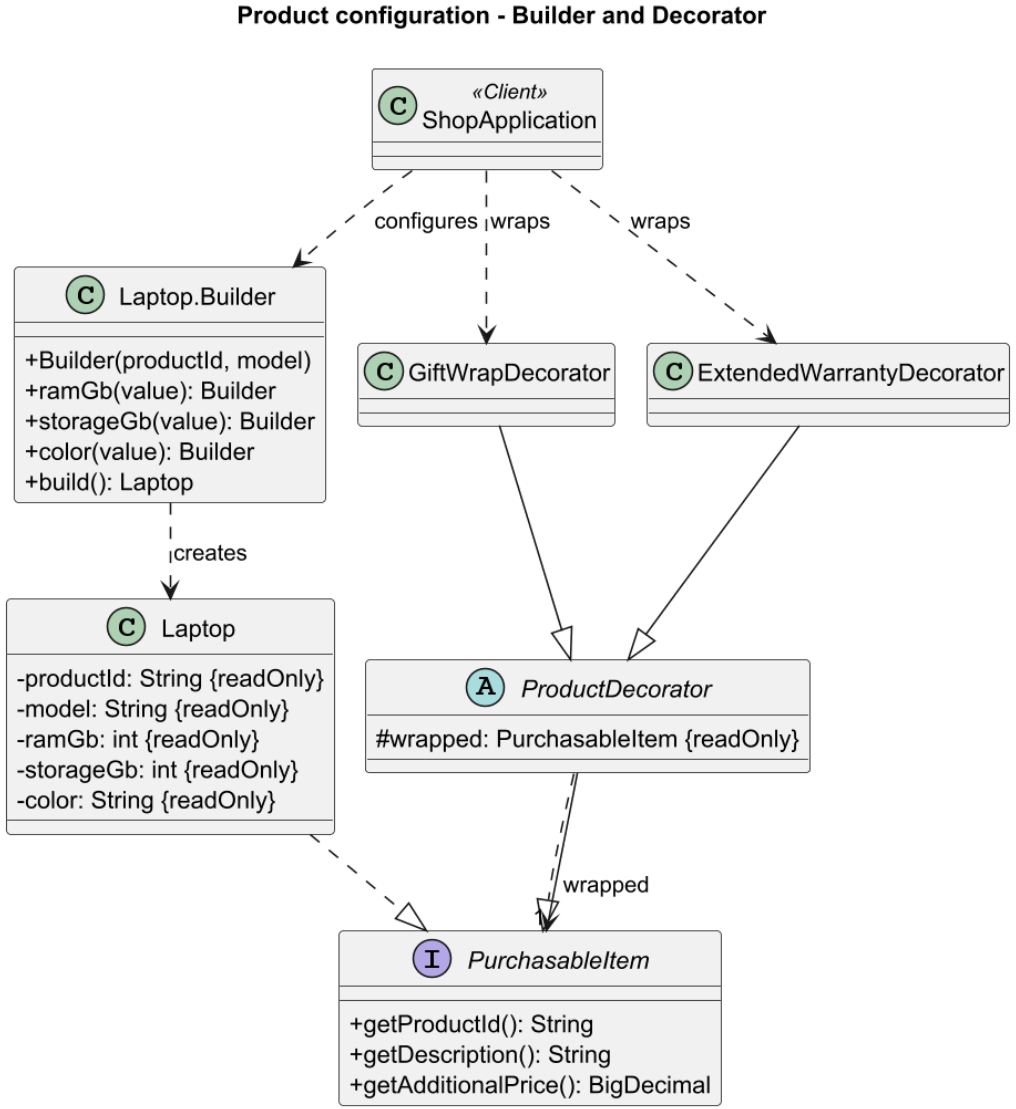


Figure 1. Builder constructs Laptop. Both service decorators wrap the shared PurchasableItem interface. Supporting value records and tests are omitted.

8 UML - catalog and payment

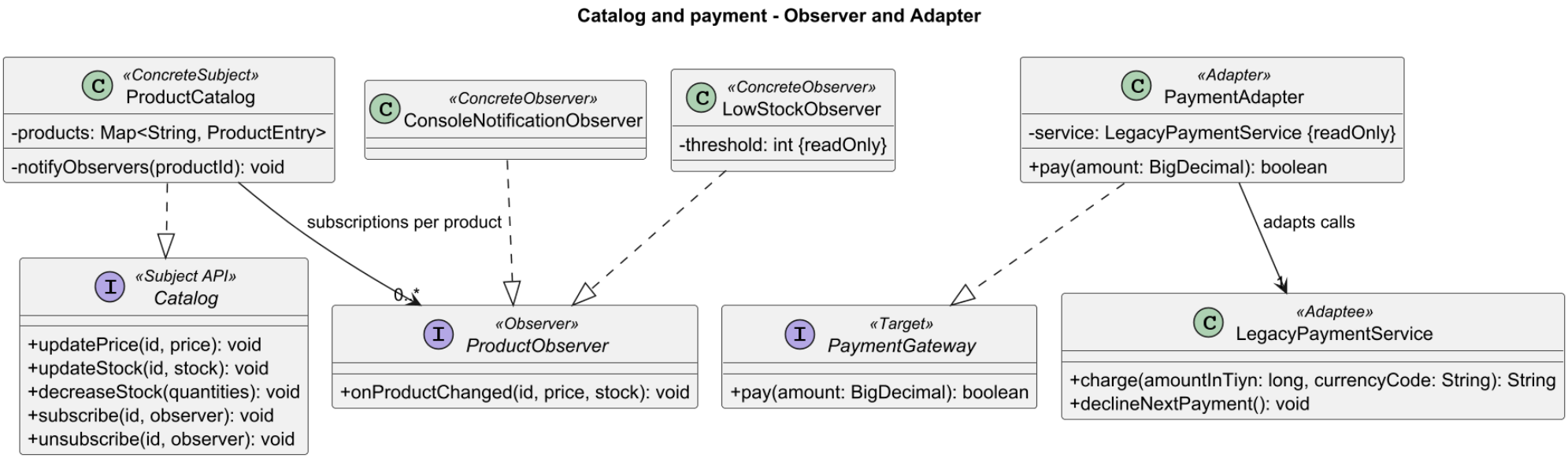


Figure 2. ProductCatalog delivers callbacks through ProductObserver. PaymentAdapter implements the target interface and holds the legacy service.

Catalog also exposes product registration and current price/stock queries. This diagram shows the operations directly involved in Observer and the payment adapter.

9 UML - cart and checkout

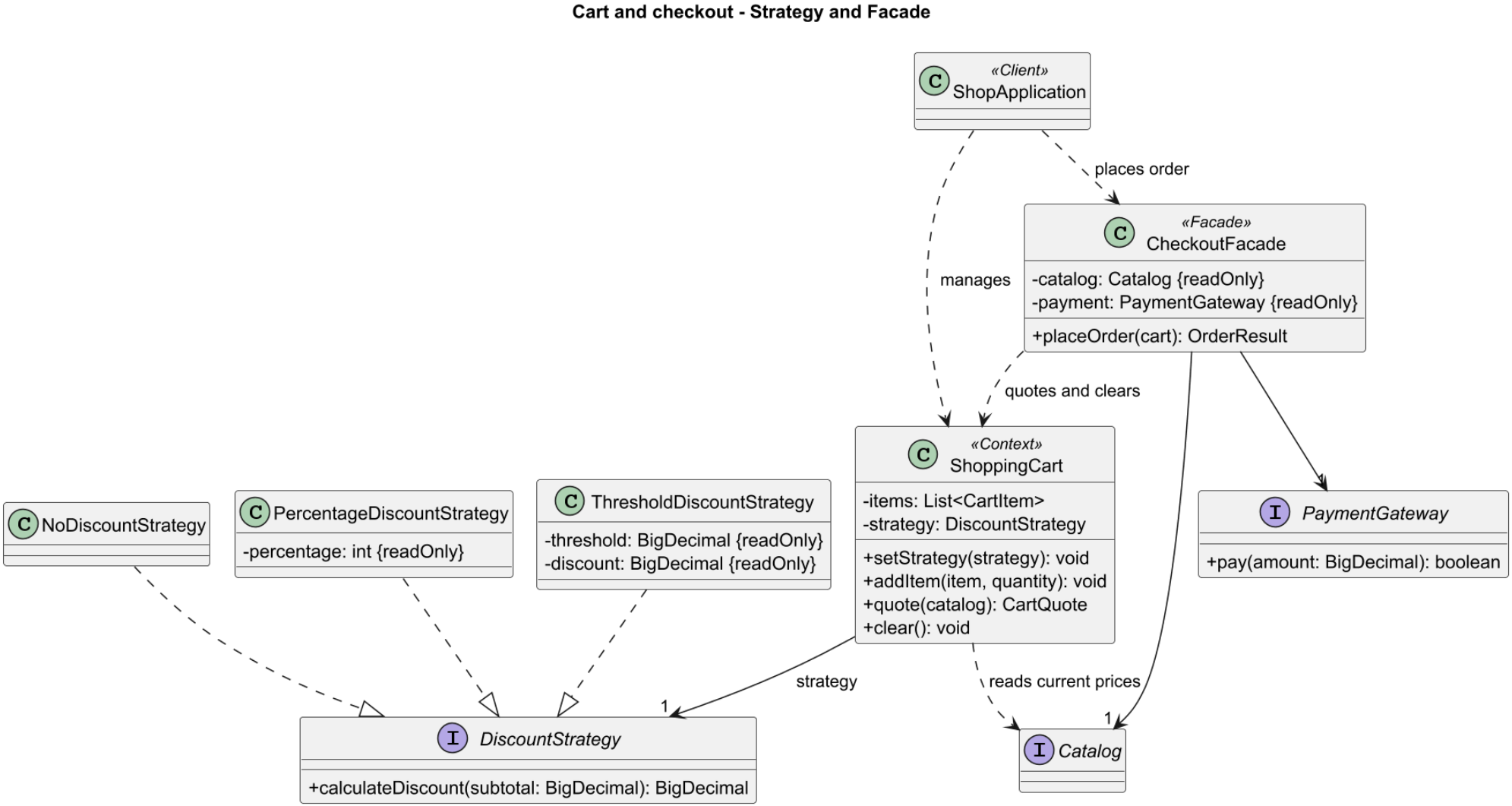


Figure 3. The cart holds a replaceable strategy. The facade holds catalog and payment references and coordinates a purchase through their interfaces.

10 UML - purchase sequence

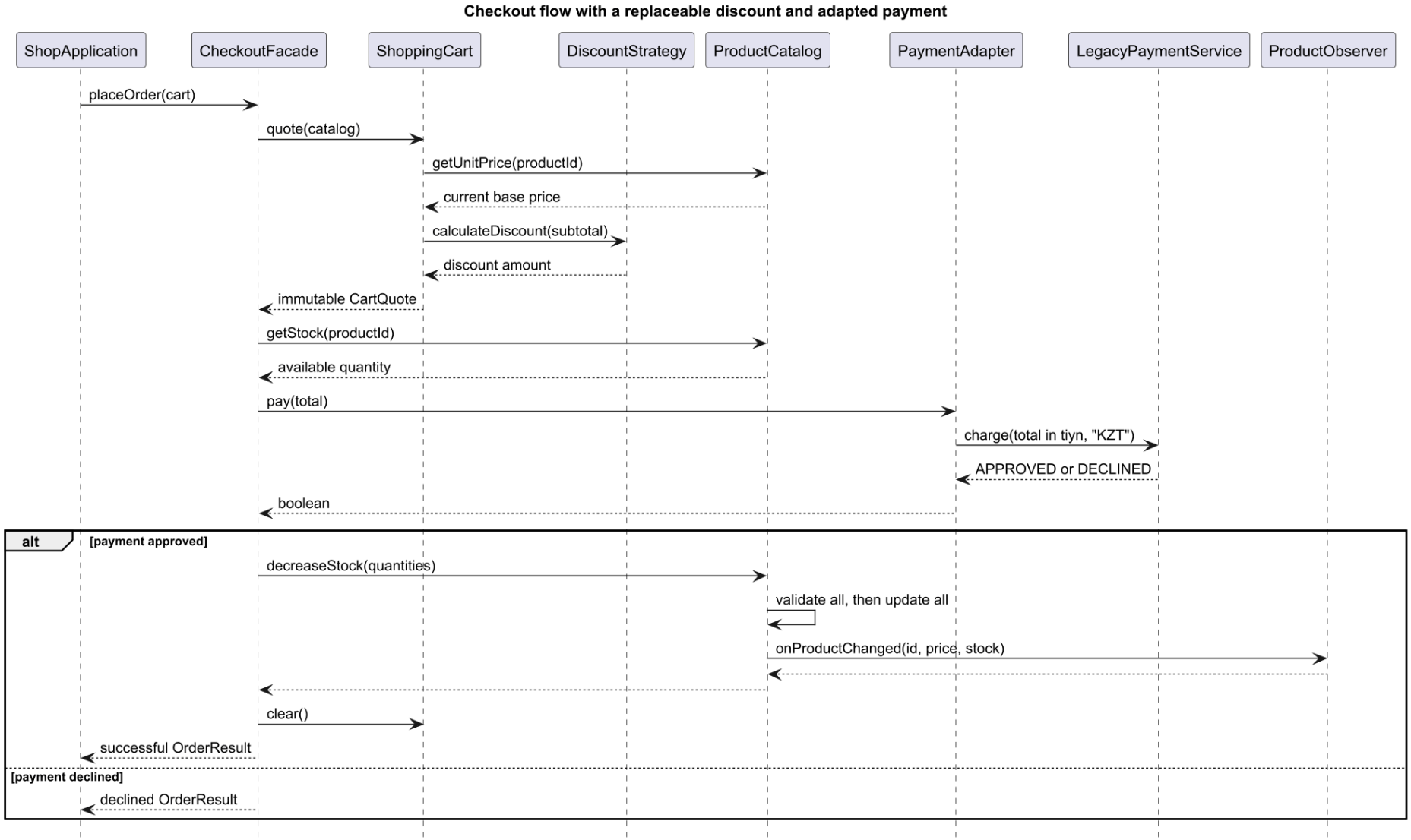


Figure 4. The sequence shows a valid-stock checkout. Insufficient stock is rejected before payment. Payment approval and decline take separate paths.

11 Application screenshots - menu and price update

The screenshots were captured from the application running in IntelliJ IDEA. The report focuses the images on console output. Full screenshots are included in docs/screenshots.

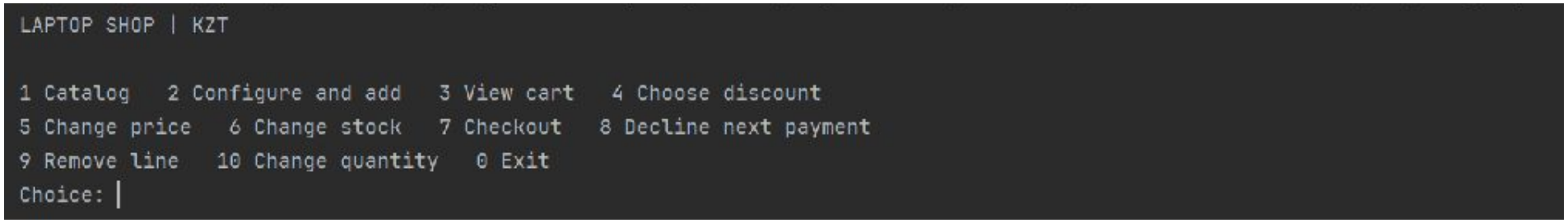


Figure 5. The interactive menu exposes product configuration, cart operations, discount replacement and catalog changes.

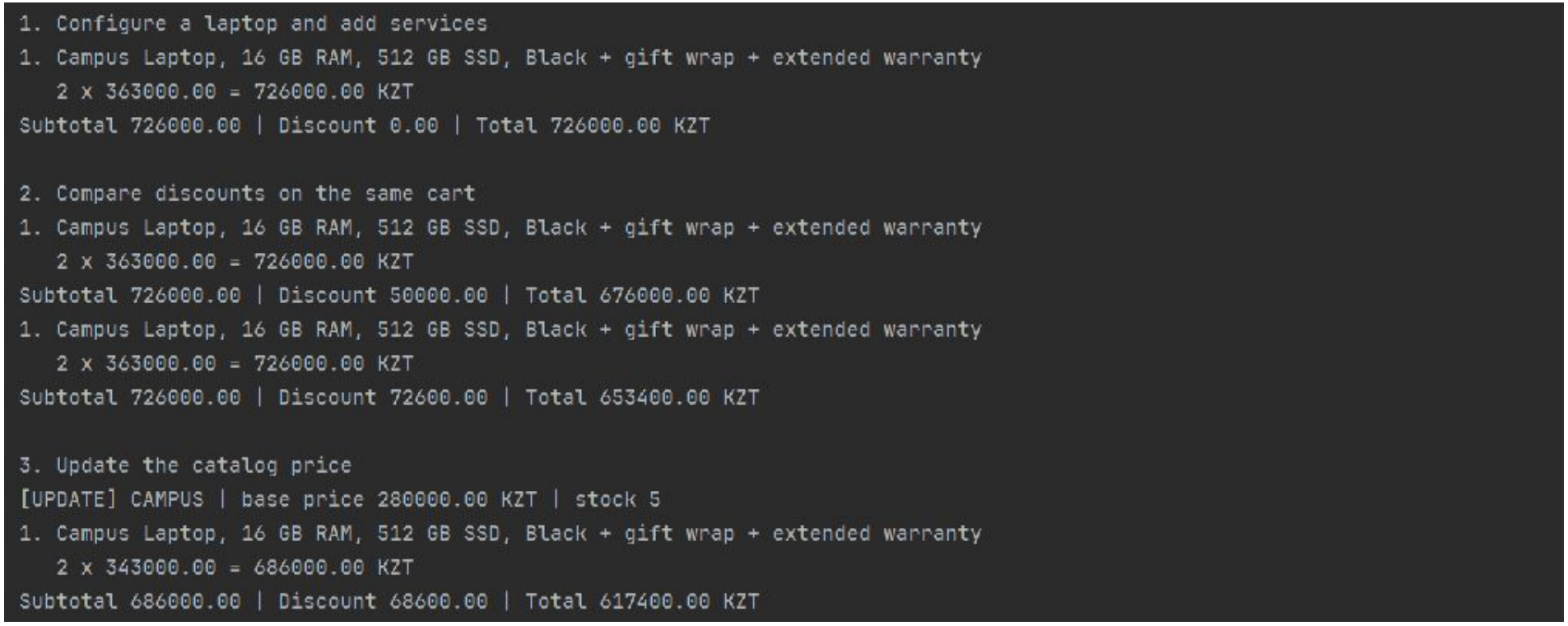


Figure 6. The sample purchase constructs a configured product, adds services and compares discount strategies on the same cart.

12 Application screenshots - checkout result

```
Subtotal 686000.00 | Discount 68600.00 | Total 617400.00 KZT

4. Decline a payment, then retry checkout
Payment declined. The cart is ready for another attempt.
1. Campus Laptop, 16 GB RAM, 512 GB SSD, Black + gift wrap + extended warranty
   2 x 343000.00 = 686000.00 KZT
Subtotal 686000.00 | Discount 68600.00 | Total 617400.00 KZT
Cart lines after checkout: 1
Stock after declined payment: 5
[UPDATE] CAMPUS | base price 280000.00 KZT | stock 3
[LOW STOCK] CAMPUS has 3 units left.
Order confirmed.
1. Campus Laptop, 16 GB RAM, 512 GB SSD, Black + gift wrap + extended warranty
   2 x 343000.00 = 686000.00 KZT
Subtotal 686000.00 | Discount 68600.00 | Total 617400.00 KZT
Cart lines after checkout: 0
Stock after successful payment: 3
Payment adapter sent 61740000 tiyn.

Process finished with exit code 0
```

Figure 7. A price notification changes the next quote. A declined payment preserves the cart and stock. The approved retry sends stock notifications and clears the cart.

The complete output is saved in docs/demo-output.txt. The purchase amounts in Section 6 are taken from this run.

13 Conclusion and further work

Conclusion

The project combines six design patterns in one working purchase flow. Builder creates a configuration, Decorator adds services, Strategy selects a discount algorithm, Observer reports catalog changes, Adapter translates payment calls and Facade coordinates checkout. The shared interfaces allow these parts to work together while keeping each pattern visible in the implementation.

Our checks confirm the normal purchase path and recoverable failures. A decline preserves the order inputs, stock validation covers all lines of the same product, and a failing display observer leaves a committed catalog update available to the remaining subscribers. Immutable quotes preserve the price used for an order.

Current limitations

Data is kept in memory and is reset at application startup. Prices use one currency. Payments are simulated. Operations and observer callbacks execute on one thread. The reentrancy guard protects a nested call to the same facade during a callback. Concurrent access across customers would require additional coordination.

Further work

A next version could persist products and completed orders, add customer accounts and expose a graphical interface. Real payment integration would require persistent payment identifiers, idempotent requests and recovery for failures between charging and committing an order. Concurrent checkout would need stock reservations or transactional inventory updates. New discount rules and service decorators can be added through the existing contracts.

References

1. AITU, Final Project Defense Requirements, course handout, pp. 1-2.
2. Eric Freeman et al., Head First Design Patterns, O'Reilly.
3. Robert C. Martin, Clean Code, Chapters 16-17, Prentice Hall, 2008.
4. JUnit 5.12.2 User Guide.
5. PlantUML Class Diagrams.

Repository and source code
https://github.com/ErknDt/SDP_Final_Project